Process Verification

Lab 7

Quentin Meneghini

22nd May 2026

Facultat d'Informàtica de Barcelona

Contents

1 Design Overview **1**

1.1 Overview 1

1.2 Module Interface 1

1.3 State Machine Design 1

1.4 Formal Verification (SVA) 3

2 Plan **4**

3 Formal Statements **5**

3.1 Logic 5

3.2 Assumptions 5

3.3 Assertions 5

4 Bug **7**

4.1 Bug 0 7

4.2 Bug 1 7

4.3 Summary 8

A Appendix Chapter **9**

A.1 Use of tools 9

A.2 Use of time 9

A.3 Use of AI 9

A.4 Output Valid 10

A.5 Output Bugged 0 11

A.6 Output Bugged 1 16

Chapter 1

Design Overview

1.1 Overview

The `packet` module implements a synchronous Finite State Machine (FSM) that models the reception of a data packet through successive processing stages: header, payload, checksum, and completion. It generates status flags (`valid_pkt`, `error_pkt`) to indicate whether a packet was successfully received or failed due to checksum errors.

1.2 Module Interface

Signal Name	Direction	Description
`clk`	Input	System clock. All state transitions occur on the rising edge.
`reset`	Input	Synchronous active-high reset. Forces the FSM to IDLE.
`start_pkt`	Input	Initiates packet reception (valid in IDLE).
`hdr_done`	Input	Signals completion of header processing.
`payload_done`	Input	Signals completion of payload processing.
`chk_ok`	Input	Indicates checksum verification passed.
`chk_fail`	Input	Indicates checksum verification failed.
`abort`	Input	Aborts reception at any stage, forcing FSM to IDLE.
`valid_pkt`	Output	Pulses high when a packet is successfully received.
`error_pkt`	Output	Pulses high when checksum fails.
`state[2:0]`	Output	Current encoded state of the FSM.

1.3 State Machine Design

The FSM consists of five states. An asynchronous `abort` signal can transition the system back to `IDLE` from any active state.

State Diagram

Every nameless arrow implies that all inputs that have not been specified translate in that state.

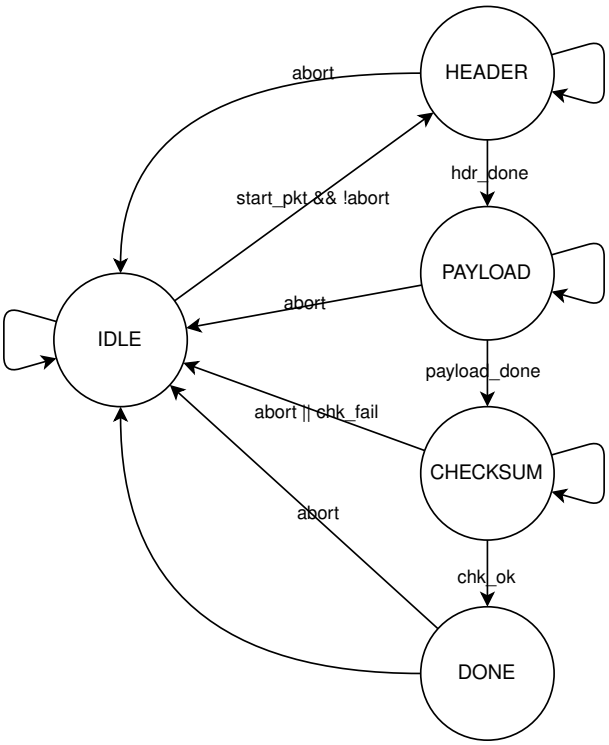

Figure 1.1: svg image

State Descriptions

- **IDLE (3'b000):** The default state. Waits for `start_pkt` to begin processing. Outputs are cleared.
- **HEADER (3'b001):** Processing the packet header. Holds state until `hdr_done` is asserted.
- **PAYLOAD (3'b010):** Processing the packet payload. Holds state until `payload_done` is asserted.
- **CHECKSUM (3'b011):** Verifying data integrity. Branches based on `chk_ok` or `chk_fail`. Assumes both cannot be asserted simultaneously.
- **DONE (3'b100):** Asserts `valid_pkt` and automatically returns to `IDLE` on the next clock cycle.

1.4 Formal Verification (SVA)

To ensure the robustness of the RTL implementation, the design has been bounded-model checked using **EBMC** and SystemVerilog Assertions (SVA). Key formal properties include:

1. **State Reachability & Hold:** Guarantees that the FSM only transitions on valid signals and holds its state otherwise.
2. **Spurious Output Prevention:** Uses the `$past()` operator to ensure `valid_pkt` and `error_pkt` only pulse high immediately following the correct checksum evaluation.
3. **Reset & Abort Compliance:** Proves that a reset or abort strictly clears outputs and returns the state space to `IDLE`, overriding any active transitions.

Chapter 2

Plan

- Understand the dut functionality and specification
- Determine the finite state machine graph (input $\rightarrow$ state)
- Determine how the graph translates into outputs (state $\rightarrow$ output)
- Determine assumption on inputs
- Determine edge cases
- Determine timing issues
- Determine assertions on outputs
- Implementation
- Debugging

Chapter 3

Formal Statements

Here we will introduce some useful logic followed by each statement, starting from the assumptions and then move on to the assertions.

3.1 Logic

`past_valid` A helper register initialized to 0 that transitions to 1 after the first clock edge. It safely gates properties that evaluate historical data via the `$past()` operator, preventing tool errors on cycle zero.

3.2 Assumptions

`initial_reset` Forces the `reset` signal to be active during the very first clock cycle (when `past_valid` is 0) to ensure the FSM initializes cleanly.

`exclusive_outcome` Constrains the solver so that `chk_ok` and `chk_fail` are mutually exclusive and never asserted simultaneously. This could have been transformed into an assertion, but it would have tested the input instead of the dut itself.

3.3 Assertions

`_reset` & `_abort` Verify that asserting a reset or an abort unconditionally forces the FSM back to the `IDLE` state on the next cycle with both output flags cleared.

`_state` Guarantees the FSM is mathematically restricted to the five valid encoded states, proving no illegal states can be reached.

State Progression (`idle_header`, `header_payload`, `payload_checksum`, `checkusm_done`, `checksum_idle`, `done_idle`): Prove that the FSM correctly transitions to the next expected state when the appropriate progression flag is asserted.

State Hold (`idle_idle`, `header_header`, `payload_payload`, `checksum_checksum`): Guarantee that the FSM retains its current state when no progression signal or abort is asserted.

Output Safety (`safe_valid_pkt`, `safe_error_pkt`): Prove that the output flags are never asserted spuriously. They only pulse high if the FSM was actively in the `CHECKSUM` state during the previous cycle with the correct outcome flag.

Output Pulse (`pulse_valid_pkt`, `pulse_error_pkt`): Verify that the output flags remain asserted for exactly one clock cycle, preventing it from a pulse behavior.

Chapter 4

Bug

Here we will introduce some bugs to test the architecture. The complete outputs can be retrieved in the appendix and will be linked promptly. [cite: 167, 168]

4.1 Bug 0

We used the `DONE` state instead of `IDLE` after a failing packet.

Original: `state <= IDLE;`

Bugged: `state <= DONE;`

The trace can be reviewed in the appendix A.5.

Analysis

The tool **REFUTE** the `checksum_idle` property. A single counter-example is given and it shows the trace that leads to a failure of the assertion. At state 10 the FSM is in the state `CHECKSUM` (011) with `chk_fail` asserted, but instead of returning to `IDLE` (000), the bug forces the FSM to become `DONE` (100).

4.2 Bug 1

Here we changed the `abort` state into a non-existent one: `3'd5`.

Original: `state <= IDLE;`

Bugged: `state <= 3'd5;`

The trace can be reviewed in the appendix A.6.

Analysis

The tool **REFUTE** two separate properties simultaneously: `_abort` and `_state`. Two distinct counterexample traces were generated, one for each property, but they both look at the same bug from different perspectives. In state 9 the FSM is in the unknown state (101), this is a violation of both properties. On one hand, the inputs that led the dut to the violation, should have returned a `DONE` state. On the other hand, that state does not even exist.

4.3 Summary

The following table summarizes the formal verification results after introducing the intentional bugs into the FSM design:

Experiment	Injected Bug	Failed Properties	Counterexamples
Bug 0	`DONE` state instead of `IDLE`	`checksum_idle`	1
Bug 1	Invalid state `3'd5` instead of `IDLE`	`_abort`, `_state`	2

Table 4.1: Summary of injected bugs and their resulting formal property violations.

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of 8 h distributed as follows.

- Understanding the problem: 1h.
- Planning: 1h.
- Implementation: 1h.
- Bugs: 3h.
- Report: 2h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer

A.4 Output Valid

```
Parsing packet.sv
Converting
Type-checking Verilog::packet
Generating Decision Problem
Using MiniSAT 2.2.1 with simplifier
Properties
Solving with propositional reduction
SAT checker: instance is UNSATISFIABLE
UNSAT: No path found within bound

** Results:
[packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
[packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
[packet._reset] always (packet.reset |=> packet.state == packet.IDLE && !packet
    .valid_pkt && !packet.error_pkt): PROVED up to bound 20
[packet._abort] always (disable iff (packet.reset) packet.abort |=> packet.
    state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): PROVED up
    to bound 20
[packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
    || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
    .state == packet.CHECKSUM || packet.state == packet.DONE): PROVED up to
    bound 20
[packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
    IDLE && packet.start_pkt && !packet.abort |=> packet.state == packet.HEADER)
    : PROVED up to bound 20
[packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
    IDLE && !packet.start_pkt && !packet.abort |=> packet.state == packet.IDLE):
     PROVED up to bound 20
[packet.header_payload] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && packet.hdr_done && !packet.abort |=> packet.state == packet
    .PAYLOAD): PROVED up to bound 20
[packet.header_header] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && !packet.hdr_done && !packet.abort |=> packet.state ==
    packet.HEADER): PROVED up to bound 20
[packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && packet.payload_done && !packet.abort |=> packet.state ==
    packet.CHECKSUM): PROVED up to bound 20
[packet.payload_payload] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && !packet.payload_done && !packet.abort |=> packet.state ==
    packet.PAYLOAD): PROVED up to bound 20
[packet.checkusm_done] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_ok && !packet.abort |=> packet.state == packet
    .DONE && packet.valid_pkt): PROVED up to bound 20
[packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_fail && !packet.abort |=> packet.state ==
    packet.IDLE && packet.error_pkt): PROVED up to bound 20
[packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort |=>
    packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
    PROVED up to bound 20
[packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
    DONE && !packet.abort |=> packet.state == packet.IDLE): PROVED up to bound
    20
[packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.error_pkt |-> !$past(packet.error_pkt)): PROVED up to bound 20
[packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
     packet.error_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_fail) && !$past(packet.abort)): PROVED up to bound 20
[packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.valid_pkt |-> !$past(packet.valid_pkt)): PROVED up to bound 20
```

```
[packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.valid_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_ok) && !$past(packet.abort)): PROVED up to bound 20
```

A.5 Output Bugged 0

```
Parsing packet.sv
Converting
Type-checking Verilog::packet
Generating Decision Problem
Using MiniSAT 2.2.1 with simplifier
Properties
Solving with propositional reduction
SAT checker: instance is SATISFIABLE
SAT: path found
SAT checker: instance is UNSATISFIABLE
UNSAT: No path found within bound

** Results:
[packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
[packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
[packet._reset] always (packet.reset |=> packet.state == packet.IDLE && !packet
    .valid_pkt && !packet.error_pkt): PROVED up to bound 20
[packet._abort] always (disable iff (packet.reset) packet.abort |=> packet.
    state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): PROVED up
    to bound 20
[packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
    || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
    .state == packet.CHECKSUM || packet.state == packet.DONE): PROVED up to
    bound 20
[packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
    IDLE && packet.start_pkt && !packet.abort |=> packet.state == packet.HEADER)
    : PROVED up to bound 20
[packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
    IDLE && !packet.start_pkt && !packet.abort |=> packet.state == packet.IDLE):
     PROVED up to bound 20
[packet.header_payload] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && packet.hdr_done && !packet.abort |=> packet.state == packet
    .PAYLOAD): PROVED up to bound 20
[packet.header_header] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && !packet.hdr_done && !packet.abort |=> packet.state ==
    packet.HEADER): PROVED up to bound 20
[packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && packet.payload_done && !packet.abort |=> packet.state ==
    packet.CHECKSUM): PROVED up to bound 20
[packet.payload_payload] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && !packet.payload_done && !packet.abort |=> packet.state ==
    packet.PAYLOAD): PROVED up to bound 20
[packet.checkusm_done] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_ok && !packet.abort |=> packet.state == packet
    .DONE && packet.valid_pkt): PROVED up to bound 20
[packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_fail && !packet.abort |=> packet.state ==
    packet.IDLE && packet.error_pkt): REFUTED
Counterexample:

Transition system state 0
----------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 1
```

```
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b111 (111)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b0

Transition system state 1
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b000 (000)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 2
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 1
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 3
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
```

```
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b000 (000)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 4
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 5
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 6
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 1
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 0
  packet.valid_pkt = 0
```

```
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 7
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b011 (011)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 8
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b011 (011)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 9
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b011 (011)
  packet.IDLE = 3'b000 (000)
```

```
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 10
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b011 (011)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 11
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 1
  packet.state = 3'b100 (100)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

[packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort |=>
    packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
    PROVED up to bound 20
[packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
    DONE && !packet.abort |=> packet.state == packet.IDLE): PROVED up to bound
    20
[packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.error_pkt |-> !$past(packet.error_pkt)): PROVED up to bound 20
[packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
     packet.error_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_fail) && !$past(packet.abort)): PROVED up to bound 20
[packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.valid_pkt |-> !$past(packet.valid_pkt)): PROVED up to bound 20
[packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
     packet.valid_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_ok) && !$past(packet.abort)): PROVED up to bound 20
```

A.6 Output Bugged 1

```
Parsing packet.sv
Converting
Type-checking Verilog::packet
Generating Decision Problem
Using MiniSAT 2.2.1 with simplifier
Properties
Solving with propositional reduction
SAT checker: instance is SATISFIABLE
SAT: path found
SAT checker: instance is SATISFIABLE
SAT: path found
SAT checker: instance is UNSATISFIABLE
UNSAT: No path found within bound

** Results:
[packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
[packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
[packet._reset] always (packet.reset |=> packet.state == packet.IDLE && !packet
    .valid_pkt && !packet.error_pkt): PROVED up to bound 20
[packet._abort] always (disable iff (packet.reset) packet.abort |=> packet.
    state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): REFUTED
Counterexample:

Transition system state 0
----------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 1
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b0

Transition system state 1
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b000 (000)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
```

```
  packet.past_valid = 1'b1

Transition system state 2
-------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 3
-------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 4
-------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 5
```

```
122 ----------------------------------------------------
123   packet.clk = ?
124   packet.reset = 0
125   packet.start_pkt = 1
126   packet.hdr_done = 0
127   packet.payload_done = 0
128   packet.chk_ok = 0
129   packet.chk_fail = 0
130   packet.abort = 0
131   packet.valid_pkt = 0
132   packet.error_pkt = 0
133   packet.state = 3'b000 (000)
134   packet.IDLE = 3'b000 (000)
135   packet.HEADER = 3'b001 (001)
136   packet.PAYLOAD = 3'b010 (010)
137   packet.CHECKSUM = 3'b011 (011)
138   packet.DONE = 3'b100 (100)
139   packet.past_valid = 1'b1
140
141 Transition system state 6
142 ----------------------------------------------------
143   packet.clk = ?
144   packet.reset = 0
145   packet.start_pkt = 0
146   packet.hdr_done = 0
147   packet.payload_done = 0
148   packet.chk_ok = 1
149   packet.chk_fail = 0
150   packet.abort = 0
151   packet.valid_pkt = 0
152   packet.error_pkt = 0
153   packet.state = 3'b001 (001)
154   packet.IDLE = 3'b000 (000)
155   packet.HEADER = 3'b001 (001)
156   packet.PAYLOAD = 3'b010 (010)
157   packet.CHECKSUM = 3'b011 (011)
158   packet.DONE = 3'b100 (100)
159   packet.past_valid = 1'b1
160
161 Transition system state 7
162 ----------------------------------------------------
163   packet.clk = ?
164   packet.reset = 0
165   packet.start_pkt = 0
166   packet.hdr_done = 1
167   packet.payload_done = 0
168   packet.chk_ok = 0
169   packet.chk_fail = 0
170   packet.abort = 0
171   packet.valid_pkt = 0
172   packet.error_pkt = 0
173   packet.state = 3'b001 (001)
174   packet.IDLE = 3'b000 (000)
175   packet.HEADER = 3'b001 (001)
176   packet.PAYLOAD = 3'b010 (010)
177   packet.CHECKSUM = 3'b011 (011)
178   packet.DONE = 3'b100 (100)
179   packet.past_valid = 1'b1
180
181 Transition system state 8
182 ----------------------------------------------------
183   packet.clk = ?
184   packet.reset = 0
```

```
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 1
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 9
----------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 1
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b101 (101)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

[packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
    || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
    .state == packet.CHECKSUM || packet.state == packet.DONE): REFUTED
Counterexample:

Transition system state 0
----------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 1
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b0

Transition system state 1
----------------------------------------------------
```

```
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b000 (000)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 2
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 1
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 3
----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 4
----------------------------------------------------
  packet.clk = ?
  packet.reset = 1
  packet.start_pkt = 0
```

```
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 5
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 1
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 0
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b000 (000)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 6
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 7
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 1
  packet.payload_done = 0
  packet.chk_ok = 0
```

```
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b001 (001)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 8
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 1
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b010 (010)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

Transition system state 9
-----------------------------------------------------
  packet.clk = ?
  packet.reset = 0
  packet.start_pkt = 0
  packet.hdr_done = 0
  packet.payload_done = 0
  packet.chk_ok = 1
  packet.chk_fail = 0
  packet.abort = 0
  packet.valid_pkt = 0
  packet.error_pkt = 0
  packet.state = 3'b101 (101)
  packet.IDLE = 3'b000 (000)
  packet.HEADER = 3'b001 (001)
  packet.PAYLOAD = 3'b010 (010)
  packet.CHECKSUM = 3'b011 (011)
  packet.DONE = 3'b100 (100)
  packet.past_valid = 1'b1

[packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
    IDLE && packet.start_pkt && !packet.abort |=> packet.state == packet.HEADER)
    : PROVED up to bound 20
[packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
    IDLE && !packet.start_pkt && !packet.abort |=> packet.state == packet.IDLE):
     PROVED up to bound 20
[packet.header_payload] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && packet.hdr_done && !packet.abort |=> packet.state == packet
    .PAYLOAD): PROVED up to bound 20
[packet.header_header] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && !packet.hdr_done && !packet.abort |=> packet.state ==
```

```
    packet.HEADER): PROVED up to bound 20
[packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && packet.payload_done && !packet.abort |=> packet.state ==
    packet.CHECKSUM): PROVED up to bound 20
[packet.payload_payload] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && !packet.payload_done && !packet.abort |=> packet.state ==
    packet.PAYLOAD): PROVED up to bound 20
[packet.checkusm_done] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_ok && !packet.abort |=> packet.state == packet
    .DONE && packet.valid_pkt): PROVED up to bound 20
[packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_fail && !packet.abort |=> packet.state ==
    packet.IDLE && packet.error_pkt): PROVED up to bound 20
[packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort |=>
    packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
    PROVED up to bound 20
[packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
    DONE && !packet.abort |=> packet.state == packet.IDLE): PROVED up to bound
    20
[packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.error_pkt |-> !$past(packet.error_pkt)): PROVED up to bound 20
[packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
     packet.error_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_fail) && !$past(packet.abort)): PROVED up to bound 20
[packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.valid_pkt |-> !$past(packet.valid_pkt)): PROVED up to bound 20
[packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
     packet.valid_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_ok) && !$past(packet.abort)): PROVED up to bound 20
```